



Generative AI Academy

Program flow

- Day 1:
 - EDA : Data Quality checks, Data insights, well articulated, deep insights
 - Structured – JSON, nested JSON, SQL
 - Unstructured: text
 - Data preprocessing with python: lists, dictionaries
 - Graphs and Plots
 - Rich and meaningful visual analysis and insights with matplotlib and seaborn
 - Azure Machine Learning
 - EDA on Azure ML
- Day 2:
 - Knowledge Graph deep dive
 - Prompt Engineering refresh
- Day 3:
 - Integration: Graph DB, Vectors DB, SQL, hybrid mechanisms
 - Mini capstone – to be presented by 12/13 Jan

Data Quality & Data Statistics for Structured and Unstructured Healthcare Data

Why Data Quality Matters in Healthcare

Clinical safety & decision accuracy

Regulatory & audit requirements

ML / GenAI sensitivity to bad data

“Garbage in → hallucinations out”

Data quality is risk management, not cleanup.

Healthcare data landscape

Structured:

claims, billing, demographics

Semi-structured:

JSON, nested JSON, FHIR

Unstructured:

clinical notes, reports

Reality: Most modern healthcare data is nested JSON + text

Typical healthcare data

Example payload:

Patient → Demographics

Encounter → timestamps

Vitals → nested objects

Conditions → arrays

Challenges

Optional fields

Deep nesting

Mixed types

Schema drift

Common data quality issues

Missing critical fields (timestamps, vitals)

Wrong data types ("120" vs 120)

Out-of-range clinical values

Duplicate patients / encounters

Inconsistent nested structures

Matplotlib and seaborn

Matplotlib

Draws lines, shapes, text, axes, ticks, legends

Graphics Engine

Foundational

Highly customizable but needs low level code

Steeper learning curve

Seaborn

Takes a dataset and plots nicely using matplotlib

Built on top of matplotlib

Beautiful default styles

Built in support for stats plots

Integrates easily with pandas

Seaborn decides what to plot and how to summarize the data;

Matplotlib decides how it is drawn and renders it to the screen.

Your Data (NumPy / Pandas)



Seaborn (optional, high-level API)



Matplotlib (low-level rendering engine)



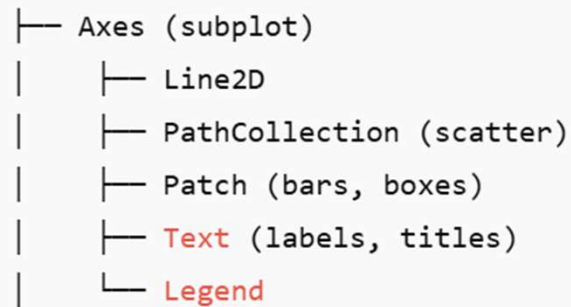
Figure → Axes → Artists → Pixels

Matplotlib

`plt.plot()`

- Get current Figure
- Get current Axes
- Create a Line2D artist
- Attach it to Axes
- Axes updates limits
- Renderer draws pixels

Figure



- **Figure:** The entire canvas (window / image)
- **Axes:** A single plot area (x-axis + y-axis)
- **Artists:** Everything that gets drawn

Seaborn

Instead of:

```
plt.scatter(df["age"], df["salary"])
```

You write:

```
sns.scatterplot(data=df, x="age", y="salary", hue="gender")
```

Seaborn:

- Accepts DataFrames
- Handles grouping, aggregation, statistics
- Maps variables to visual channels automatically

Seaborn – does statistical transformations

Seaborn does data processing before plotting.

Example:

```
sns.barplot(data=df, x="department", y="salary")
```

Behind the scenes:

- Group by department
- Compute mean salary
- Compute confidence intervals
- Pass computed values to Matplotlib

Matplotlib itself does none of this.

Seaborn – does nice styling

Seaborn sets:

- Color palettes
- Grid styles
- Font sizes
- Tick directions
- Background aesthetics

Under the hood:

`sns.set_theme()` → modifies Matplotlib rcParams

Seaborn – end to end processing

```
sns.boxplot(data=df, x="department", y="salary")
```

Step 1: Input interpretation

- Detect categorical vs numeric
- Validate column names
- Infer plot semantics

Step 2: Statistical computation

- Group data by department
- Compute quartiles, whiskers, outliers

Step 3: Create Matplotlib objects

- Create a Figure (if not exists)
- Create an Axes
- Create Patch artists for boxes
- Create Line2D for whiskers

Step 4: Styling

- Apply color palette
- Set linewidths
- Configure grid & ticks

Step 5: Render

- Matplotlib renderer converts artists → pixels
- Output to screen or file

Seaborn for plots and stats, matplotlib for fine-grained control

You still use Matplotlib for:

- Custom annotations
- Secondary y-axes
- Subplot grids beyond defaults
- Exact control of ticks, spines, limits
- Exporting figures precisely (DPI, formats)

Axis titles

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style="whitegrid")

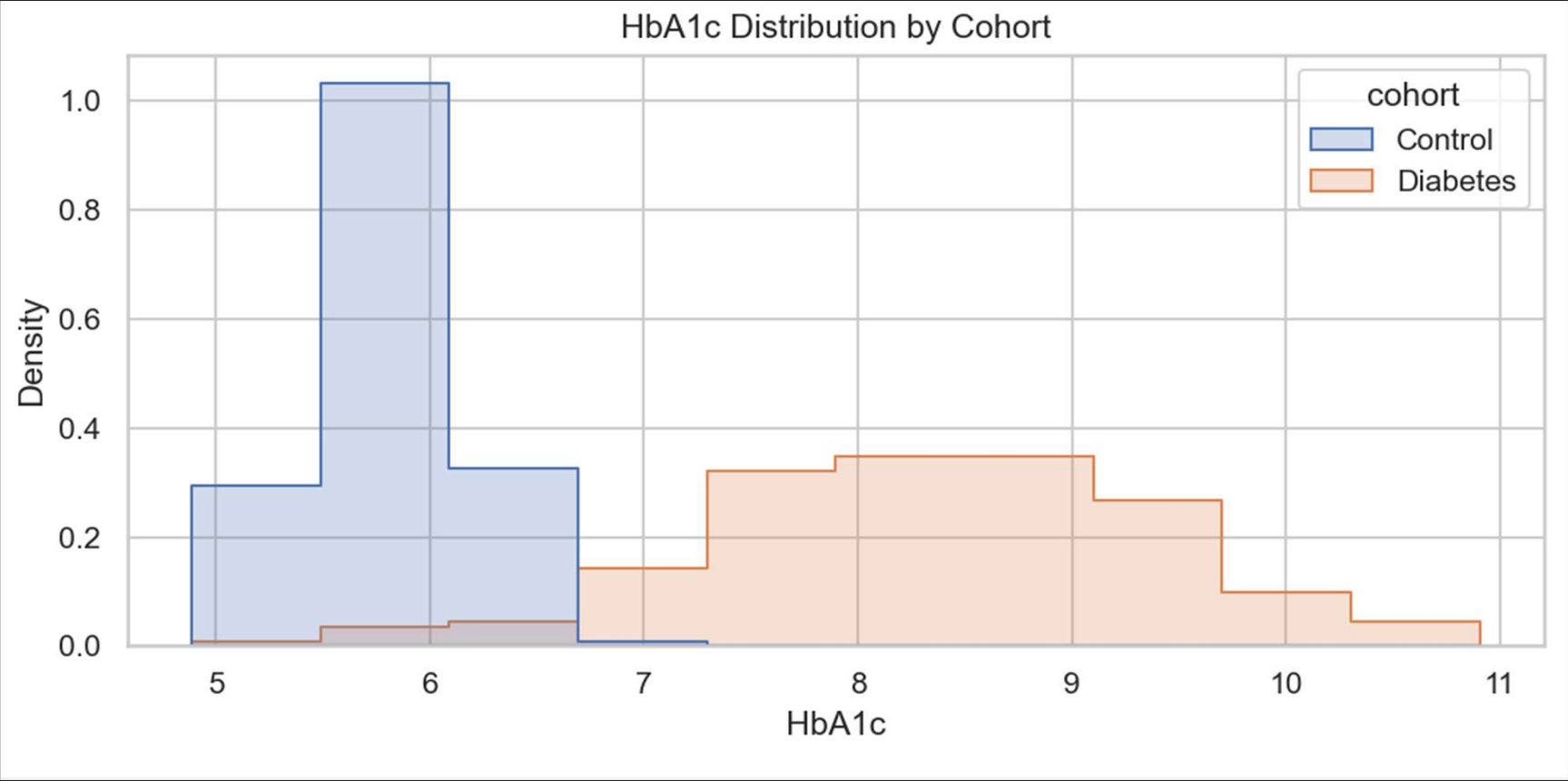
ax = sns.lineplot(data=df, x="date", y="revenue")

ax.set_title("Revenue Trend")
ax.set_ylabel("USD (Millions)")
plt.tight_layout()
plt.show()
```

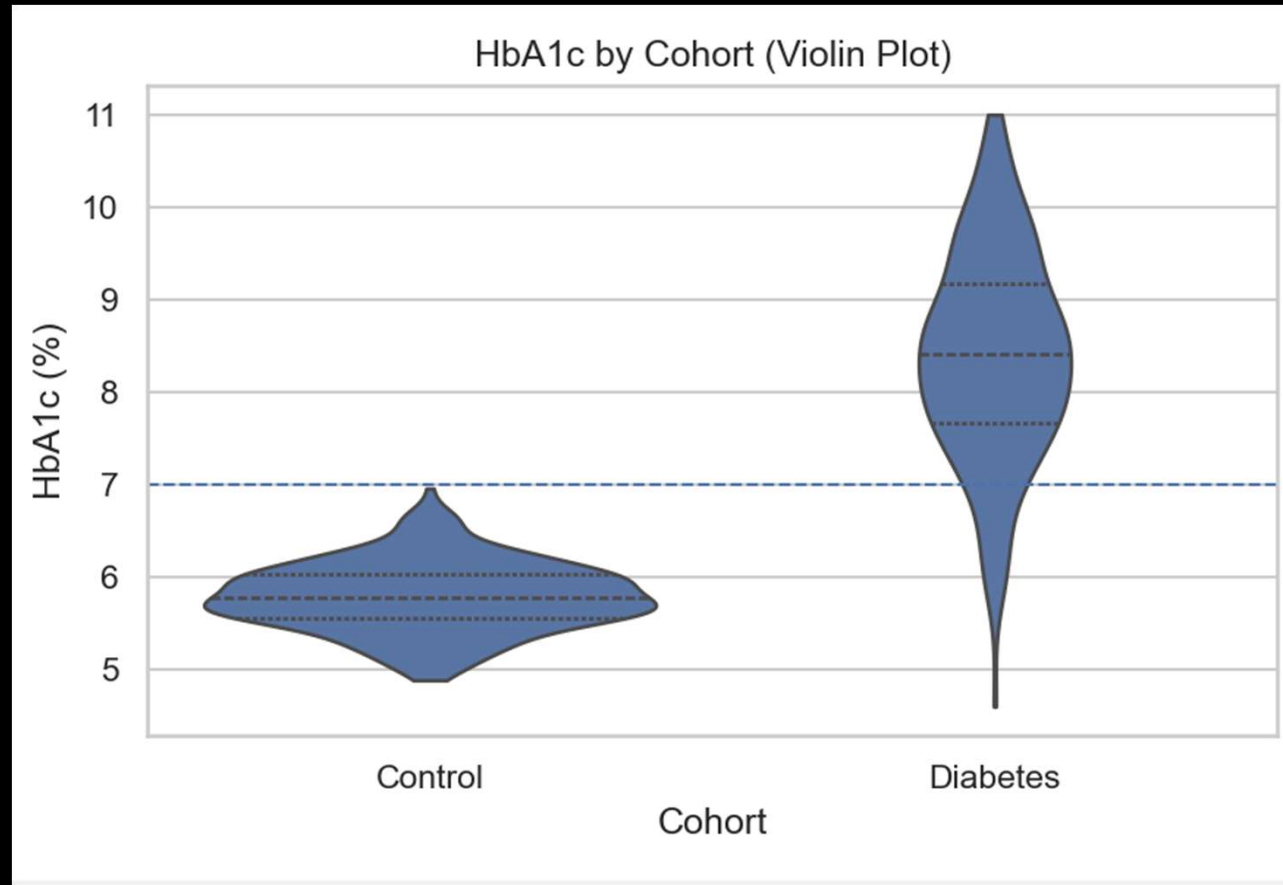
Draw a vertical line to indicate mean

```
ax = sns.histplot(df["price"], bins=50)
ax.axvline(df["price"].mean(), color="red")
```

Interpret this chart



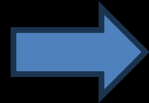
Interpret this chart



Enterprise Machine Learning with Microsoft Azure

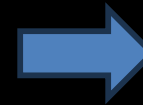
Data ingestion

- **Azure Data Factory**
Low code/no code tool:
Visual pipeline designer, control flows, event triggers, scheduling
- Data ingestion, ETL at scale
- Integrates with Databricks, Synapse
- For async http calls, Python asyncio, aiohttp are suitable



Data Eng., model training

- **Azure Databricks**
data cleaning, feature engineering at scale with Apache Spark
- Distributed Model training with Apache Spark. Integrates with Hadoop
- Notebooks
- Metadata, Delta Tables



Data Storage

- Parquet files in Azure Data Lake or AWS S3
- Azure Synapse
If you require complex sql queries, reporting or ad-hoc analysis on structured data